# IKB21403 – Vulnerability Analysis

## Week 3: BlueMoon System Hacking

*Practical Report / Walkthrough*

Name: Sara Ayra
Student ID: 52215125545

## 1. Introduction

This report documents my Week 3 practical for IKB21403 – Vulnerability Analysis using the VulnHub BlueMoon vulnerable machine. The task was to perform system hacking in a controlled lab environment and prepare a walkthrough/report.

The goal was to understand the full process rather than just capture the final flag. I identified the target on the virtual network, checked the services that were running, enumerated the web application, found information that could be used to obtain access, and then worked through privilege escalation until root access was achieved.

All testing was performed against the BlueMoon virtual machine in my own lab environment.

## 2. Lab Environment

- Attacker machine: Kali Linux
- Target machine: BlueMoon 2021 (VulnHub)
- Target IP identified during the lab: 192.168.100.177
- Network: local lab network (192.168.100.x)

The two machines were on the same network so Kali could reach BlueMoon directly. All commands below were run from my Kali terminal.

## 3. Objective

System hacking, as covered in the Week 3 material, is a process of reconnaissance, scanning, gaining access, privilege escalation, maintaining access and clearing tracks. For this task I focused on the BlueMoon machine and documented the main steps used to reach root.

The work was done only on the intentionally vulnerable machine provided for the exercise.

## 4. Reconnaissance and Port Scanning

I scanned the local network to find live hosts, and identified BlueMoon at 192.168.100.177 among the discovered addresses. I then ran an Nmap service/version scan against that address:

```
nmap -sC -sV 192.168.100.177
```

The scan showed three open TCP ports:

- 21/tcp – FTP (vsftpd 3.0.3)

- 22/tcp – SSH (OpenSSH 7.9p1)
- 80/tcp – HTTP (Apache httpd 2.4.38), title "BlueMoon:2021"

This gave three directions to investigate further: FTP for files, SSH for remote login, and HTTP for web enumeration.

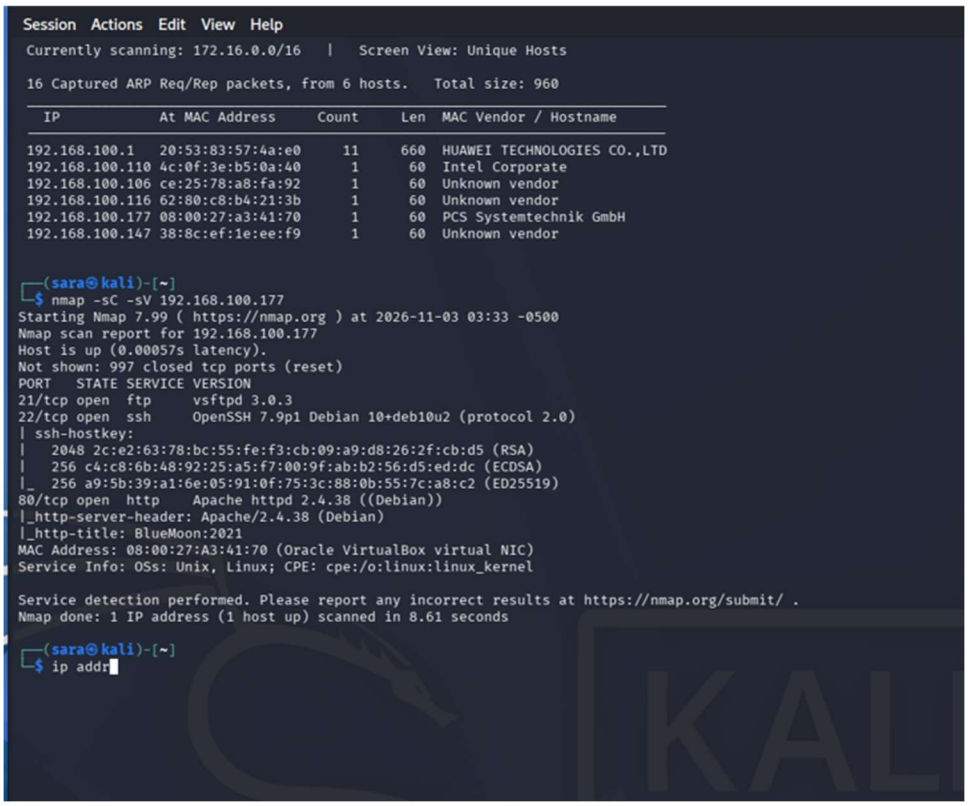



*Figure 1. Host discovery and Nmap scan identifying FTP, SSH and HTTP on 192.168.100.177.*

## 5. Web Enumeration

I opened the target in the browser on port 80 and confirmed the web server was running. The page displayed a BlueMoon-themed welcome message.

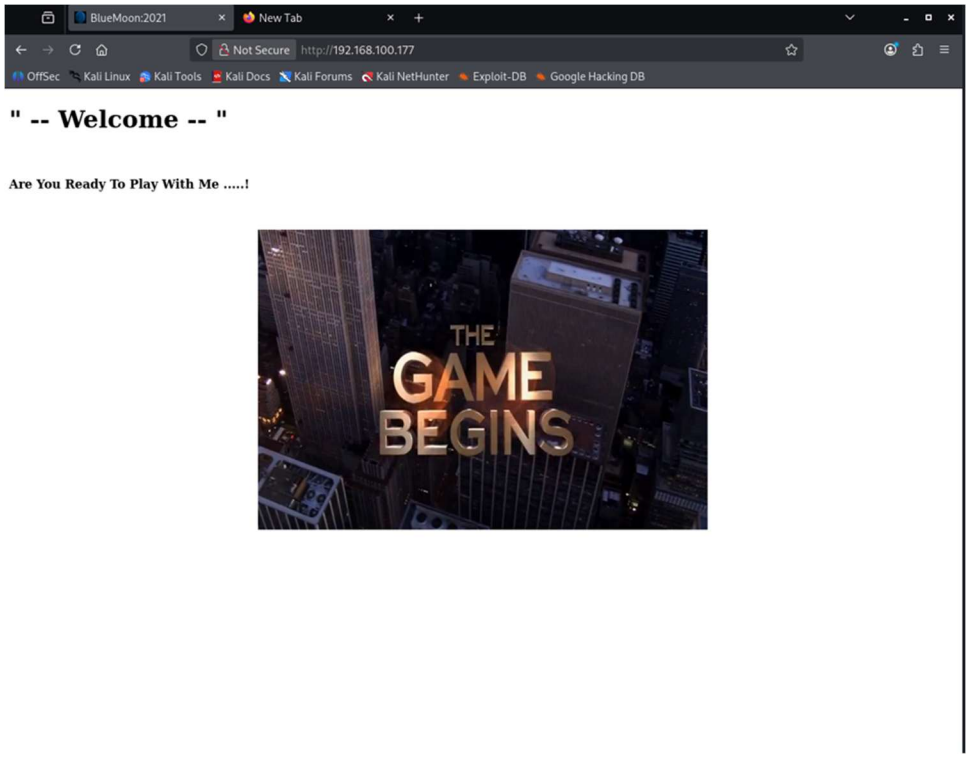



*Figure 2. BlueMoon web page reachable on port 80.*

I then checked a hidden path, /hidden_text, which returned a maintenance-style page rather than a 404.

```
curl -s http://192.168.100.177/hidden_text
```

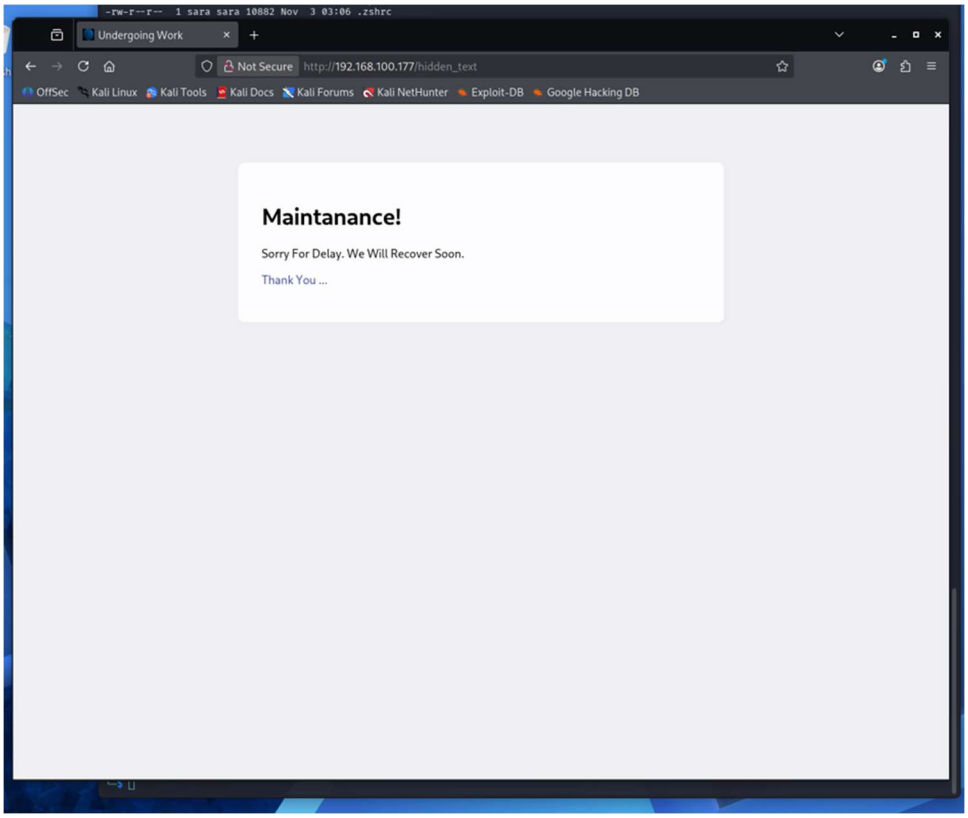



*Figure 3. The /hidden_text page returned a maintenance message.*

Viewing the page source revealed a link to a hidden QR code image, .QR_C0d3.png, referenced in an anchor tag at the bottom of the page.

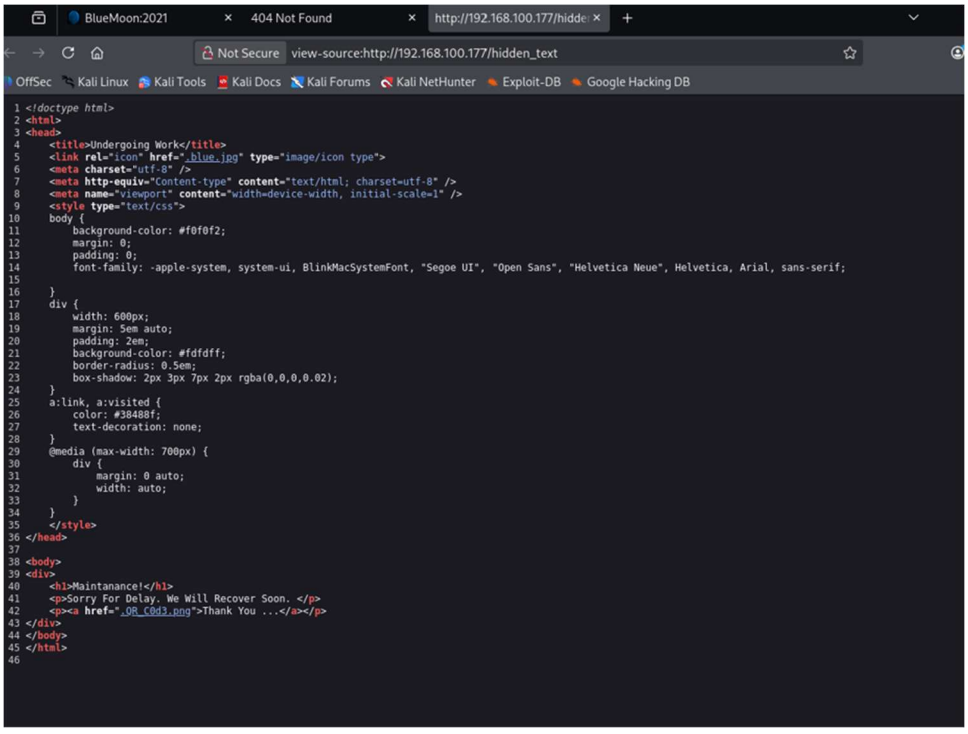


*Figure 4. Page source of /hidden_text showing the hidden QR image link.*

## 6. QR Code and FTP Information

I installed zbar-tools, downloaded the hidden QR image directly from the web server, and decoded it with zbarimg:

```
sudo apt update sudo apt install zbar-tools curl -s
http://192.168.100.177/hidden_text | grep -i qr wget
http://192.168.100.177/.QR_C0d3.png zbarimg .QR_C0d3.png
```

The decoded QR content was a small shell script embedding FTP connection details: a host placeholder, the username userftp, and a password. I used these to connect to the FTP service.




*Figure 5. zbarimg decoding the hidden QR code to reveal FTP credentials.*

## 7. FTP Access

Using the credentials from the QR code, I logged into the FTP service. The directory listing contained two files, information.txt and p_lists.txt, which I downloaded to my local machine.

```
ftp 192.168.100.177 Name: userftp Password: ******** ftp> ls ftp> get information.txt
ftp> get p_lists.txt ftp> bye
```

information.txt indicated that the account robin had a weak password and pointed to the password list for cracking. The actual FTP password and file contents are not reproduced here beyond what is needed to explain the steps.

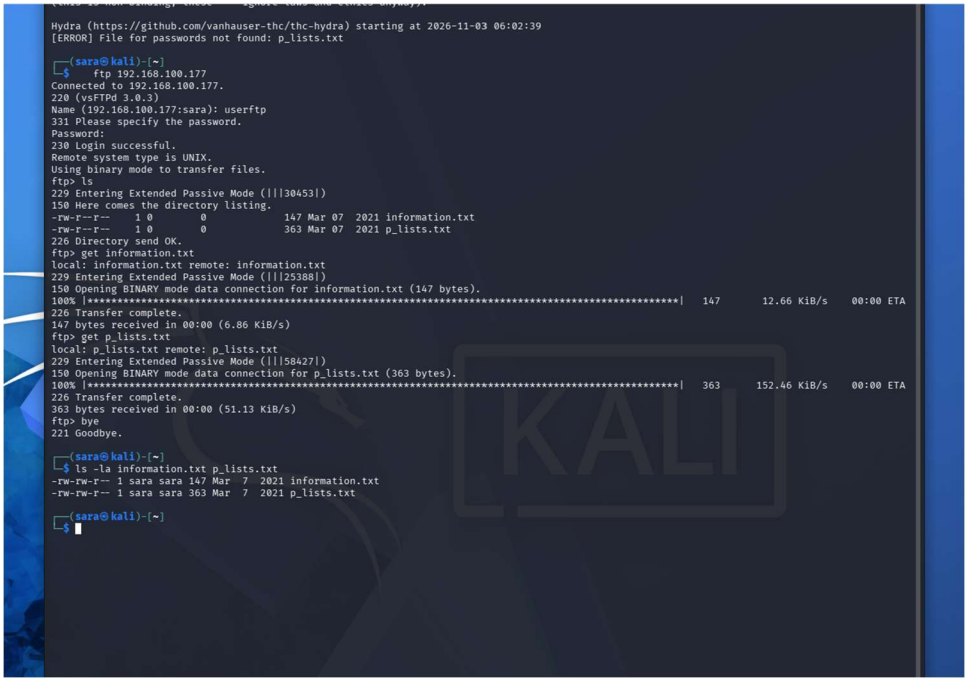



*Figure 6. FTP login and download of information.txt and p_lists.txt.*

## 8. SSH Access and User Flag

With the password list obtained from FTP, I used Hydra to brute-force the SSH login for the robin account. The scan needed the password-list path corrected once (the file had to be re-fetched into the working directory first), after which Hydra completed successfully and returned one valid credential pair.

```
hydra -l robin -P p_lists.txt -t 4 ssh://192.168.100.177
```

Hydra reported a successful match for the robin account.

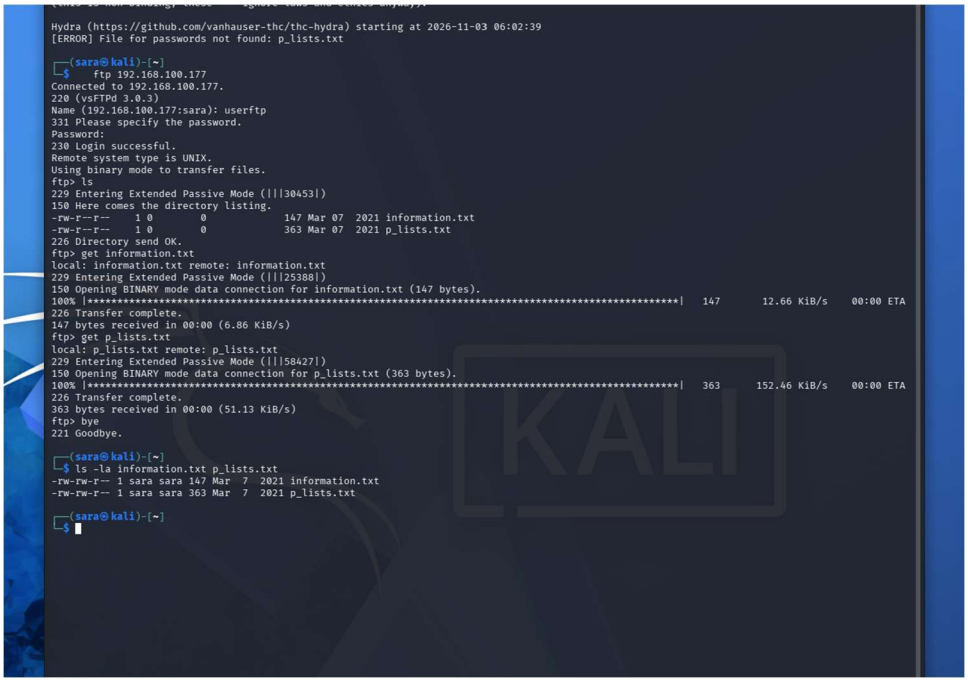



*Figure 7. Hydra found a valid SSH login for robin.*

I then connected to BlueMoon over SSH as robin and checked the home directory, where user1.txt confirmed the first user-level flag had been obtained.

```
ssh robin@192.168.100.177 $ ls $ cat user1.txt
```

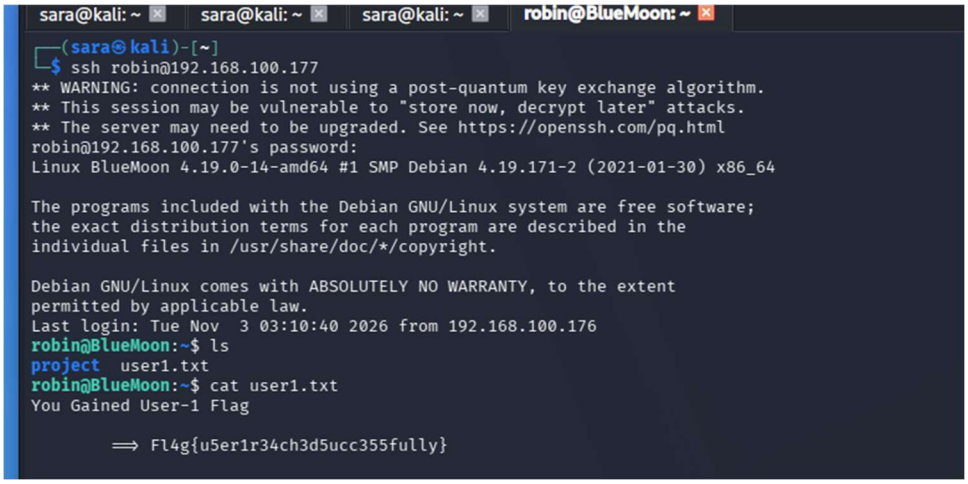



*Figure 8. SSH login as robin and the user-1 flag.*

User flag obtained: Fl4g{u5er1r34ch3d5ucc355fully}

## 9. Privilege Escalation from Robin to Jerry

With a shell as robin, I checked sudo permissions using sudo -l. The output showed that robin could run /home/robin/project/feedback.sh as the user jerry without a password.

```
sudo -l ... User robin may run the following commands on bluemoon:     (jerry)
NOPASSWD: /home/robin/project/feedback.sh
```

Inspecting feedback.sh showed that it prompts for a name and a line of feedback, then executes the feedback input directly without sanitising it. Supplying /bin/bash as the feedback input caused the script to spawn a shell running as jerry.

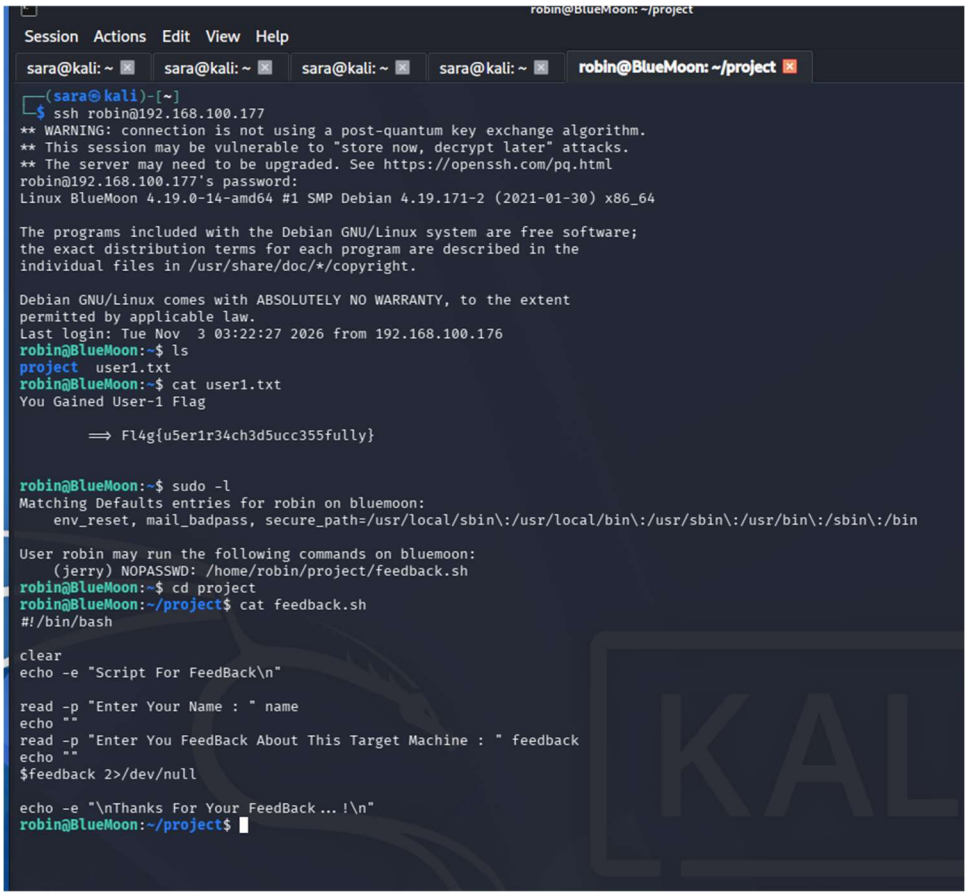



*Figure 9. sudo -l output and the contents of feedback.sh.*

Running the script and entering /bin/bash at the feedback prompt dropped me into a shell as jerry, confirmed with whoami and id. The id output also showed jerry is a member of the docker group.




*Figure 10. Breaking out of feedback.sh into a jerry shell; jerry is a member of the docker group.*

## 10. Docker Group and Root Escalation

Because jerry belongs to the docker group, Docker could be used to mount the host filesystem inside a container and obtain root on the underlying host. I ran the following command to start an Alpine container with the host root filesystem mounted, then chrooted into it:

```
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
```

Inside the container shell, whoami returned root and id confirmed UID 0. Moving to /root and listing the directory showed root.txt, which contained the final flag.

```
# whoami root # id uid=0(root) gid=0(root) ... # cd /root # ls -la # cat root.txt
```

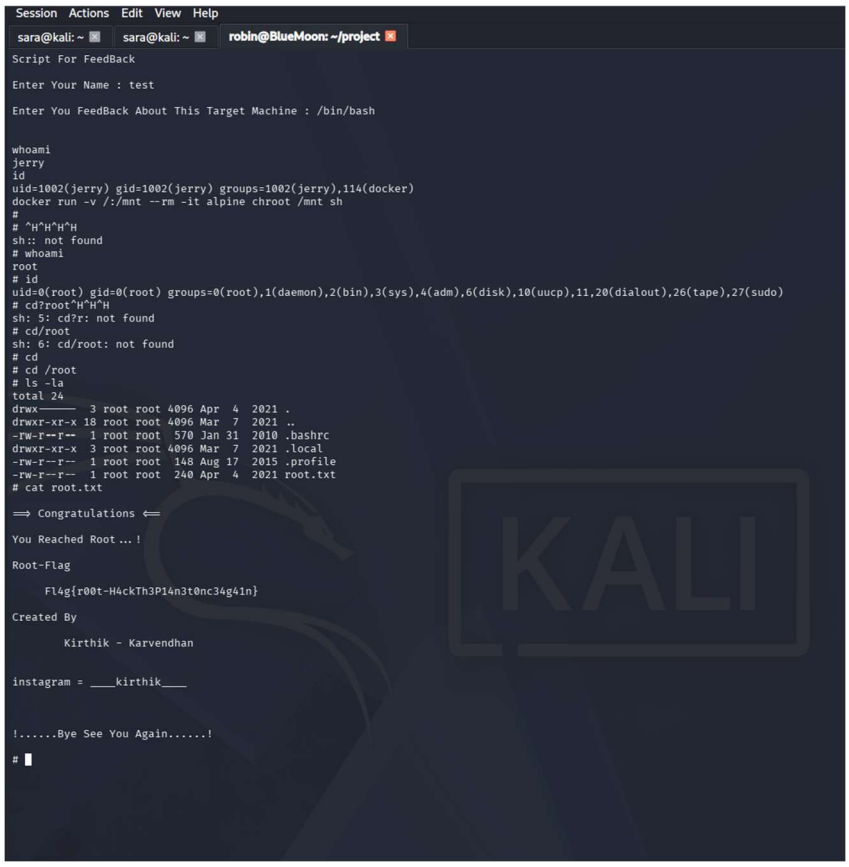



*Figure 11. Root shell obtained via Docker group abuse, with root.txt captured.*

Root flag obtained: Fl4g{r00t-H4ckTh3P14n3t0nc34g41n}

## 11. Security Findings and Recommended Mitigations

The practical showed several weaknesses that would matter on a real system:

1. Exposed FTP service and sensitive files

- Disable FTP if it is not required.
- Use SFTP/SSH or FTPS when file transfer is required.
- Do not store credentials or password lists in publicly reachable locations.

2. Weak/reusable passwords

- Use long, unique passwords.
- Add rate limiting and account lockout controls where appropriate.
- Use MFA for important remote-access services.

3. Sensitive information hidden in a web page

- Do not place credentials or secrets in HTML comments, hidden paths or downloadable files.
- Apply proper authentication and authorization to sensitive resources.

4. Unsafe sudo permission

- Do not allow a normal user to run an interactive or user-controlled script as another account without a password.
- Restrict sudo rules to the minimum required command and validate all input.

5. Docker group privilege

- Treat membership of the docker group as highly privileged, effectively root-equivalent.
- Do not give untrusted users access to the Docker socket.
- Consider rootless containers and stronger container isolation where suitable.

## 12. Conclusion

The BlueMoon lab helped me understand the full flow of a basic system-hacking exercise. The most useful part was seeing how different pieces of information connected together: the open web service led to a hidden resource, the hidden resource led to a QR code with FTP information, the FTP files led to SSH access, and the local permissions eventually led to root access.

I also learned that small mistakes are a normal part of practical work — for example, needing to re-fetch the password list into my working directory before Hydra could use it, and installing zbar-tools before I could decode the QR code. Checking the output carefully at each stage made the overall process easier to follow.

The exercise was completed in the intended controlled lab environment.

## 13. References


- IKB21403 Vulnerability Analysis – Week 3 lecture/task material.
- VulnHub – BlueMoon (2021) vulnerable machine.
- Tools used during the lab: Nmap, curl, zbarimg, FTP client, Hydra, SSH and Docker.